

Prueba de Caja Blanca

SPRINT I

REQ001

*“Título proyecto sistema de automatización de mensajes
e ingreso de datos para fechas importantes”*

Integrantes:
Alejandro De La Cruz
Santiago Nogales
Ian Escobar

Fecha 2025-08-04

Prueba caja blanca de (REQ001) "Seguridad al ingreso"

1. CÓDIGO FUENTE

```
bool iniciarSesion() {
    std::string usuario, contrasena;
    int intentos = 0;
    const int maxIntentos = 3;

    std::cout << "\n" << std::string(50, '=') << std::endl;
    std::cout << "          INICIO DE SESIÓN" << std::endl;
    std::cout << std::string(50, '=') << std::endl;

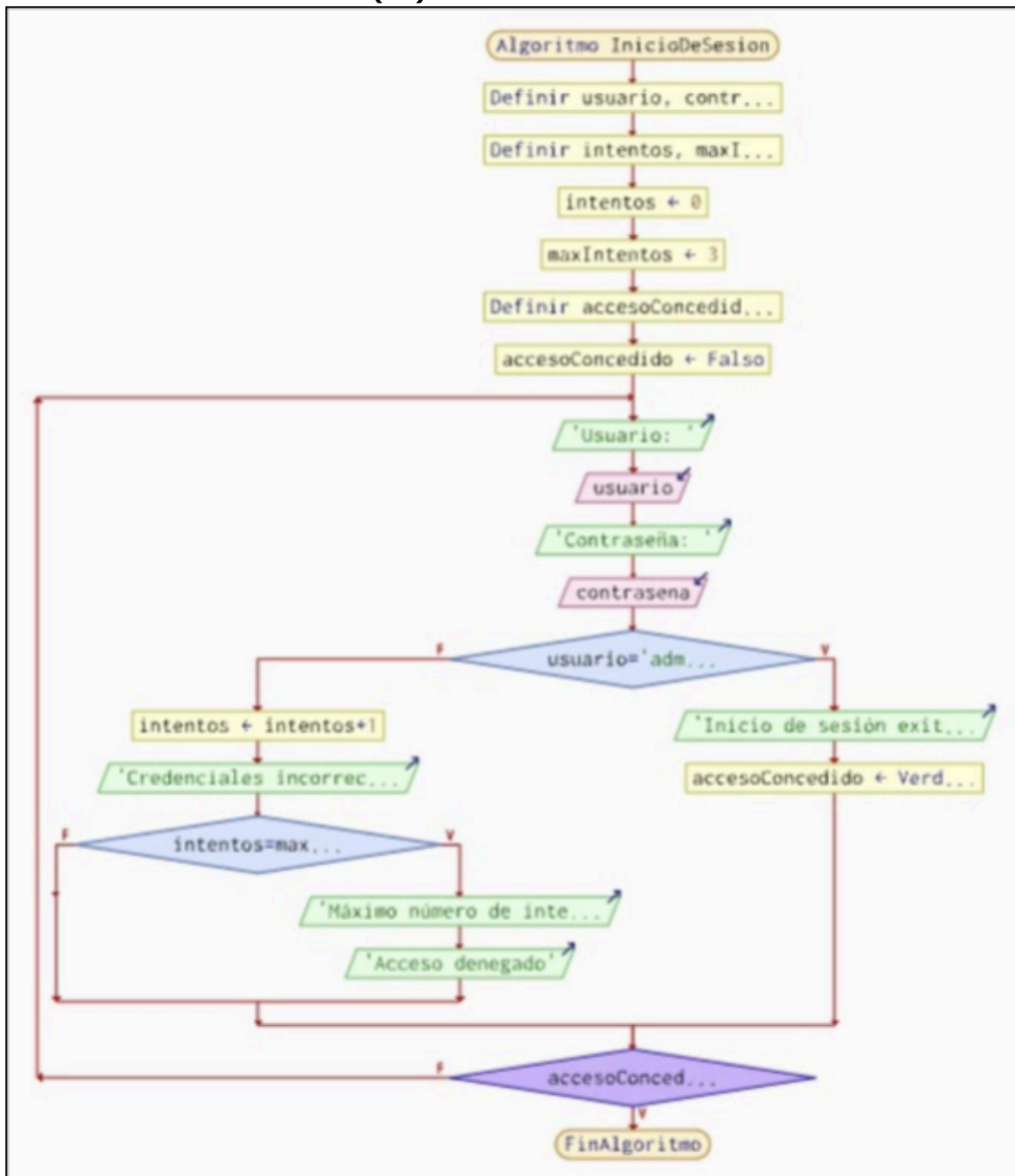
    while (intentos < maxIntentos) {
        std::cout << "\nUsuario: ";
        std::cin >> usuario;

        std::cout << "Contraseña: ";
        std::cin >> contrasena;

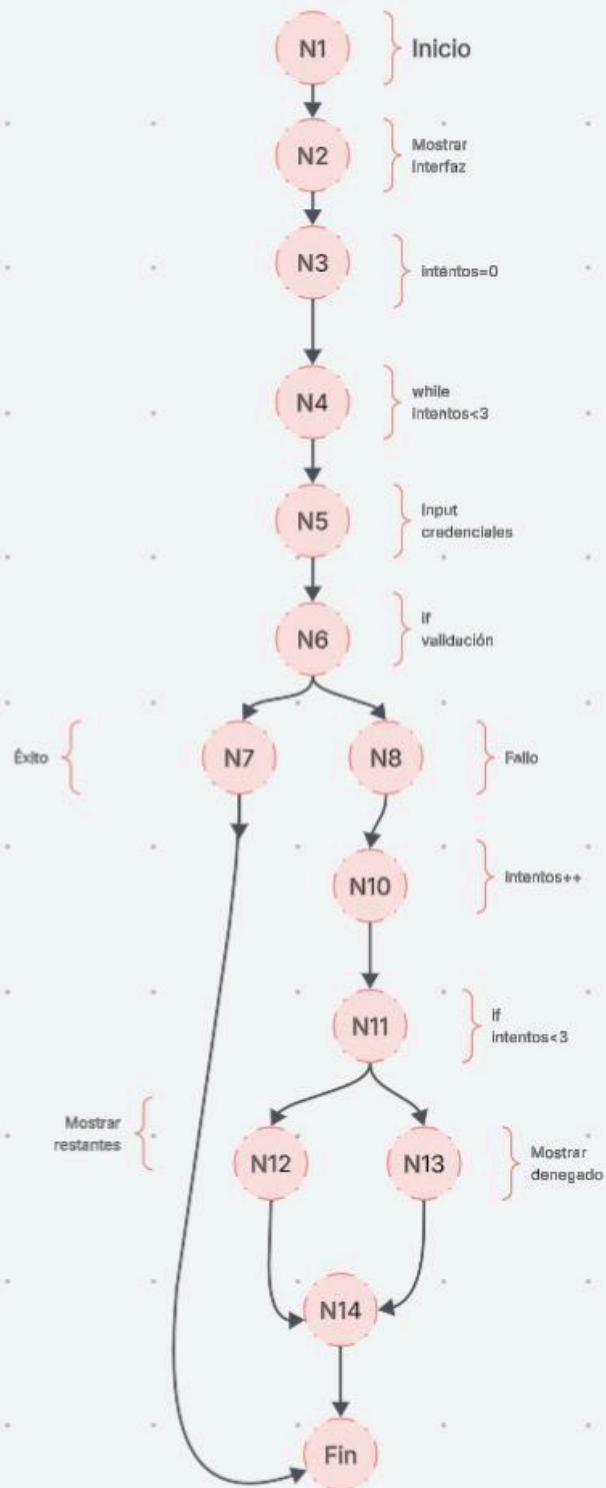
        if (usuario == "administrador" && contrasena == "administrador") {
            std::cout << "\n¡Inicio de sesión exitoso!" << std::endl;
            std::cout << "Bienvenido al Sistema de Mensajería Automática" << std::endl;
            return true;
        } else {
            intentos++;
            std::cout << "\nCredenciales incorrectas. ";
            if (intentos < maxIntentos) {
                std::cout << "Intentos restantes: " << (maxIntentos - intentos) << std::endl;
            } else {
                std::cout << "Máximo número de intentos alcanzado." << std::endl;
                std::cout << "Acceso denegado." << std::endl;
            }
        }
    }

    return false;
}
```

2. DIAGRAMA DE FLUJO (DF)



3. GRAFO DE FLUJO (GF)



4. IDENTIFICACIÓN DE LAS RUTAS (Camino básico)

RUTAS

1. R1 (Éxito c. pñímci i·tc·to): $N1 \rightarrow N2 \rightarrow N3 \rightarrow N4 \rightarrow N5 \rightarrow N6 \rightarrow N7 \rightarrow N9$
2. R2 (Éxito en segundo intento): $N1 \rightarrow N2 \rightarrow N3 \rightarrow N4 \rightarrow N5 \rightarrow N6 \rightarrow N8 \rightarrow N10 \rightarrow N11 \rightarrow N12 \rightarrow N4 \rightarrow N5 \rightarrow N6 \rightarrow N7 \rightarrow N9$
3. R« (Éxito c. tcíccí i·tc·to): Lo mismo que R2, pero el ciclo $\rightarrow N4 \rightarrow \dots$ se repite una vez más.
4. R4 (lallo c. los « i·tc·tos): $N1 \rightarrow N2 \rightarrow N3 \rightarrow N4 \rightarrow N5 \rightarrow N6 \rightarrow N8 \rightarrow N10 \rightarrow N11 \rightarrow N12 \rightarrow N4 \rightarrow N5 \rightarrow N6 \rightarrow N8 \rightarrow N10 \rightarrow N11 \rightarrow N13 \rightarrow N14$

5. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

1. $V(G) = \text{número de nodos predcados(decisiones)} + 1$

$$V(G) = 3 (N4, N6, N11) + 1 = 4$$

2. Método aristas-nodos:

$$V(G) = 15 \text{ aristas} - 13 \text{ nodos} + 2 = 4$$

DONDE:

P: Número de nodos predcado

A: Número de aristas

N: Número de nodos

SPRINT I

REQ002

Prueba caja blanca de describa el requisito funcional

1. CÓDIGO FUENTE

Pegar el trozo de código fuente que se requiere para el caso de prueba

```
void ejecutar() {
    if (!iniciarSesion()) return;

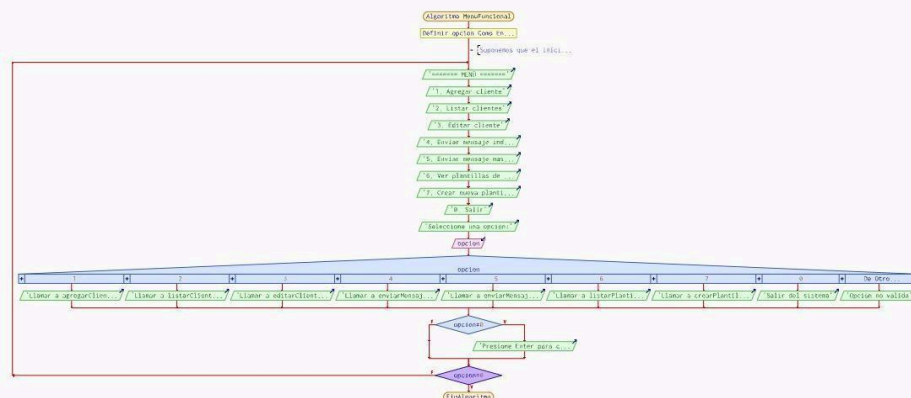
    int opcion;
    do {
        std::cout << "\n" << std::string(50, '=') << std::endl;
        std::cout << "    SISTEMA DE MENSAJERÍA AUTOMÁTICA" << std::endl;
        std::cout << std::string(50, '=') << std::endl;
        std::cout << "1. Agregar cliente\n";
        std::cout << "2. Listar clientes\n";
        std::cout << "3. Editar cliente\n";
        std::cout << "4. Enviar mensaje individual\n";
        std::cout << "5. Enviar mensaje masivo\n";
        std::cout << "6. Ver plantillas de mensajes\n";
        std::cout << "7. Crear nueva plantilla\n";
        std::cout << "0. Salir\n";
        std::cout << std::string(50, '-') << std::endl;
        std::cout << "Seleccione una opción: ";
        std::cin >> opcion;

        switch (opcion) {
            case 1: agregarCliente(); break;
            case 2: listarClientes(); break;
            case 3: editarCliente(); break;
            case 4: enviarMensajeIndividual(); break;
            case 5: enviarMensajeMasivo(); break;
            case 6: listarPlantillas(); break;
            case 7: crearPlantilla(); break;
            case 0: std::cout << "¡Gracias por usar el sistema de mensajería!\n"; break;
            default: std::cout << "Opción no válida. Intente de nuevo.\n";
        }

        if (opcion != 0) {
            std::cout << "\nPresione Enter para continuar...";
            std::cin.ignore();
            std::cin.get();
        }
    } while (opcion != 0);
}
```

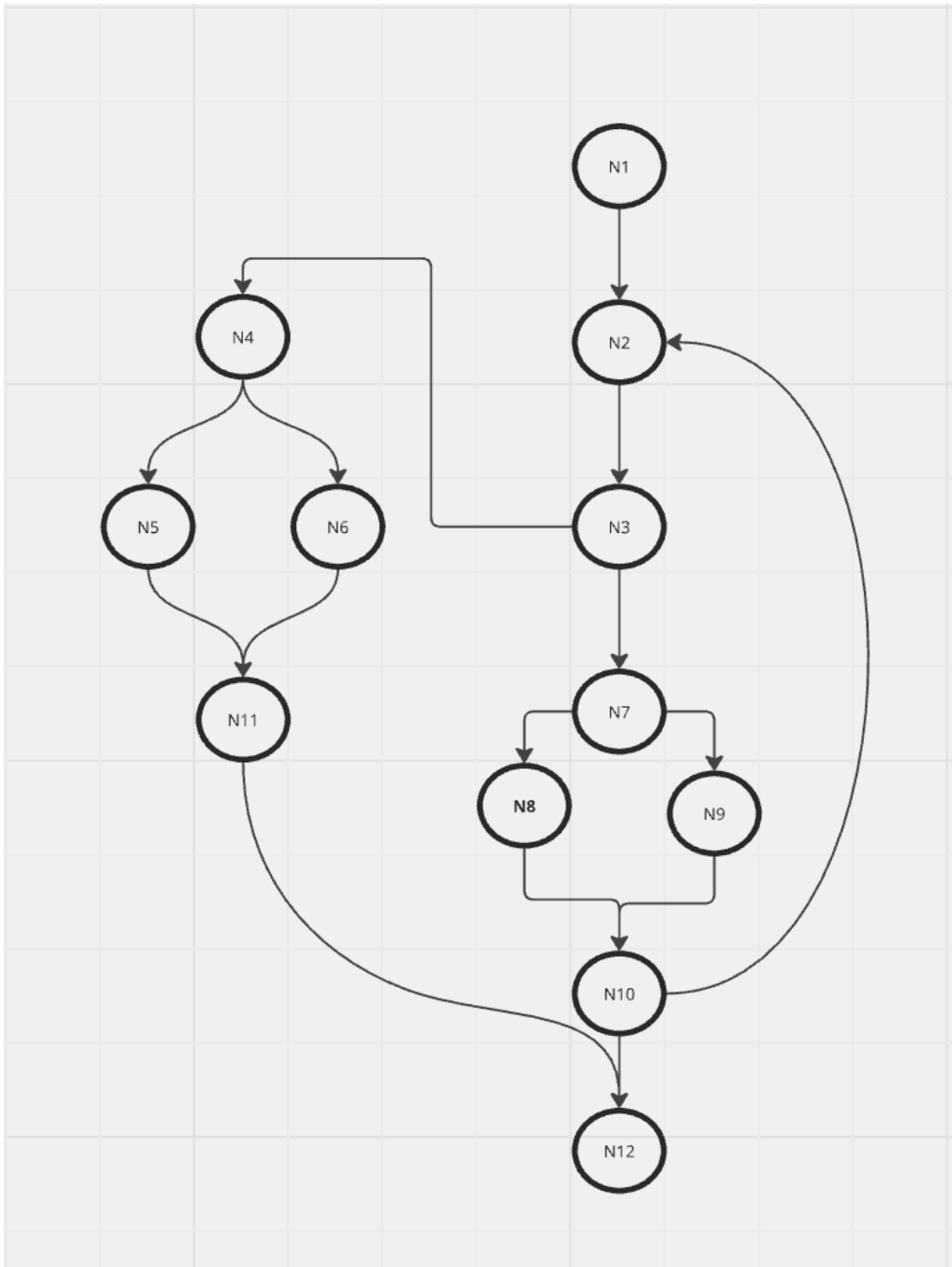
2. DIAGRAMA DE FLUJO (DF)

Realizar un DF del código fuente del numeral 1



3. GRAFO DE FLUJO (GF)

Realizar un GF en base al DF del numeral 2



4. IDENTIFICACIÓN DE LAS RUTAS (Camino basico)

Determinar en base al GF del numeral 4

Rutas Independientes:

R1: 1 → 2 → 6 (Login fallido).

R2: 1 → 2 → 3 → 4 → 5 → 6 (Login exitoso → Salir).

R3: 1 → 2 → 3 → 4 → 5 → 7 → 8 → 9/10/11/12 → 3... (Bucle de opciones).

5. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = \text{número de nodos predichados(decisiones)} + 1$
- $V(G) = P + 1 = 3 + 1 = 4$
- $V(G) = A - N + 2$
- $V(G) = A - N + 2 = 14$

DONDE:

P: Número de nodos predichado

A: Número de aristas

N: Número de nodos

SPRINT I

REQ003

Prueba caja blanca de describa el requisito funcional

1. CÓDIGO FUENTE

Pegar el trozo de código fuente que se requiere para el caso de prueba

```
void editarCliente() {
    if (clientes.empty()) {
        std::cout << "No hay clientes registrados.\n";
        return;
    }

    listarClientes();
    int clienteId;
    std::cout << "\nIngrese el ID del cliente a editar: ";
    std::cin >> clienteId;

    auto clienteIt = std::find_if(clientes.begin(), clientes.end(),
        [clienteId](const Cliente& c) { return c.id == clienteId; });

    if (clienteIt == clientes.end()) {
        std::cout << "Cliente no encontrado.\n";
        return;
    }

    std::cout << "\n=== EDITAR CLIENTE ===\n";
    std::cout << "Cliente actual:\n";
    std::cout << "Nombre: " << clienteIt->nombre << "\nTeléfono: " << clienteIt->telefono
        << "\nEmail: " << clienteIt->email << "\nEstado: " << clienteIt->estado << std::endl;

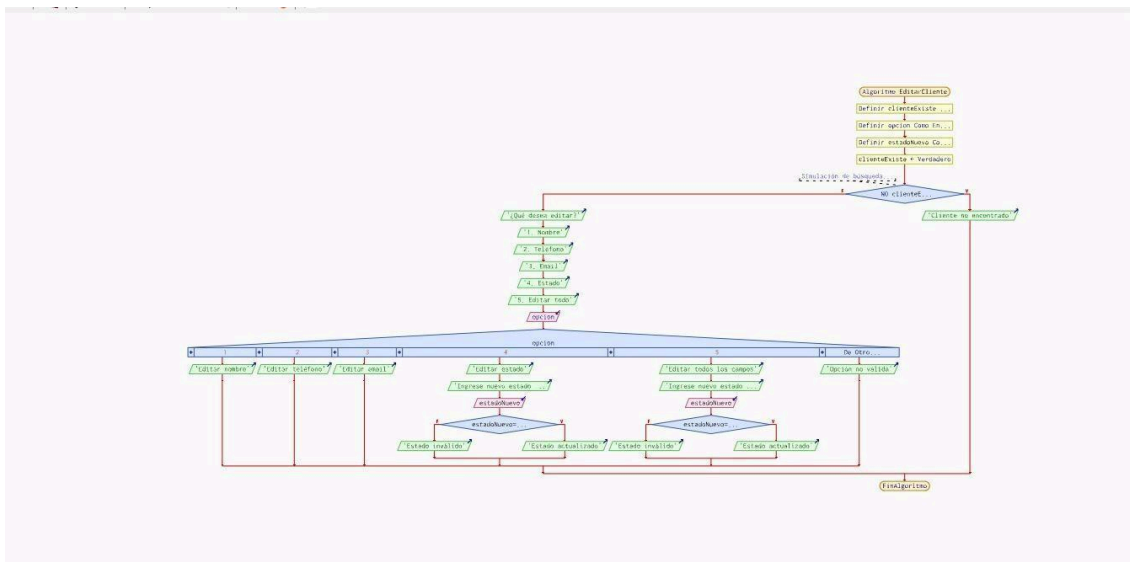
    int opcion;
    std::cout << "\n¿Qué desea editar?\n1. Nombre\n2. Teléfono\n3. Email\n4. Estado\n5. Editar todo\nSeleccione una opción: ";
    std::cin >> opcion;
    std::string nuevoValor;
    std::cin.ignore();

    switch (opcion) {
        case 1:
            std::cout << "Nuevo nombre: ";
            std::getline(std::cin, nuevoValor);
            clienteIt->nombre = nuevoValor;
            break;
        case 2:
            std::cout << "Nuevo teléfono: ";
            std::getline(std::cin, nuevoValor);
            clienteIt->telefono = nuevoValor;
            break;
        case 3:
            std::cout << "Nuevo email: ";
            std::getline(std::cin, nuevoValor);
            clienteIt->email = nuevoValor;
            break;
        case 4:
            std::cout << "Nuevo estado (activo/inactivo): ";
            std::getline(std::cin, nuevoValor);
            if (nuevoValor == "activo" || nuevoValor == "inactivo") {
                clienteIt->estado = nuevoValor;
            } else {
                std::cout << "Estado inválido. Debe ser 'activo' o 'inactivo'. \n";
            }
            break;
        case 5:
            std::cout << "Nuevo nombre: ";
            std::getline(std::cin, clienteIt->nombre);
            std::cout << "Nuevo teléfono: ";
            std::getline(std::cin, clienteIt->telefono);
            std::cout << "Nuevo email: ";
            std::getline(std::cin, clienteIt->email);
            std::cout << "Nuevo estado (activo/inactivo): ";
            std::getline(std::cin, nuevoValor);
            if (nuevoValor == "activo" || nuevoValor == "inactivo") {
                clienteIt->estado = nuevoValor;
            }
            break;
        default:
            std::cout << "Opción no válida.\n";
            return;
    }

    std::cout << "Datos del cliente actualizados exitosamente.\n";
}
```

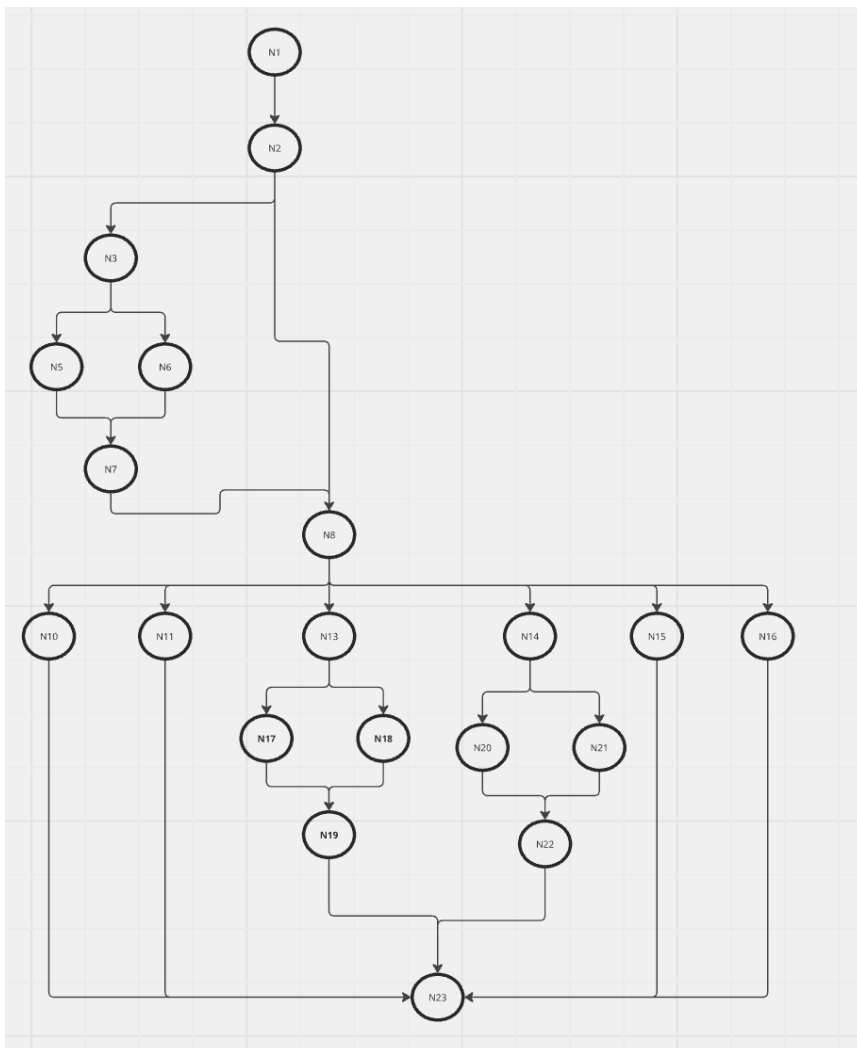
2. DIAGRAMA DE FLUJO (DF)

Realizar un DF del código fuente del numeral 1



3. GRAFO DE FLUJO (GF)

Realizar un GF en base al DF del numeral 2



4. IDENTIFICACIÓN DE LAS RUTAS (Camino basico)

Determinar en base al GF del numeral 4

RUTAS

1. **R1:** N5 → N14
o Camino: Seleccionar opción 0 (Salir) → Terminar programa.
2. **R2:** N5 → N15
o Camino: Ingresar opción inválida → Mostrar error → Volver al menú.
3. **R3:** N5 → N6 → N16 → N17 → N5
o Camino: Opción 1 (Agregar cliente) → Ejecutar función → Pausa → Volver al menú.
4. **R4:** N5 → N7 → N18 → N5
o Camino: Opción 2 (Listar clientes) → Ejecutar → Volver al menú.
5. **R5:** N5 → N9 → N20 → N21 → N5
o Camino: Opción 4 (Mensaje individual) → Enviar → Pausa → Volver al menú.

Se puede calcular de las siguientes formas:

$$A = 21$$

$$N = 18$$

- $V(G) = \text{número de nodos predichados(decisiones)} + 1$
 $V(G) = P = 8 + 1 = 9$
- $V(G) = A - N + 2$
 $V(G) = 21 - 18 + 2 = 5.$

DONDE:

P: Número de nodos predichado

A: Número de aristas

N: Número de nodos

```

Java
package controllers;

import models.Cliente;
import models.Mensaje;
import services.ClienteService;
import services.MensajeService;
// ... other imports

@Controller
public class WebController {

    @Autowired
    private ClienteService clienteService;

    @Autowired
    private MensajeService mensajeService;

    @PostMapping("/editar-cliente")
    public String editarCliente(@ModelAttribute Cliente cliente, RedirectAttributes redirectAttributes) {
        if (clienteService.editarCliente(cliente)) {
            redirectAttributes.addFlashAttribute("success", "Cliente editado exitosamente");
        } else {
            redirectAttributes.addFlashAttribute("error", "Error al editar el cliente");
        }
        return "redirect:/clientes"; // Adjust as needed
    }

    @PostMapping("/borrar-cliente/{id}")
    public String borrarCliente(@PathVariable int id, RedirectAttributes redirectAttributes) {
        if (clienteService.borrarCliente(id)) {
            redirectAttributes.addFlashAttribute("success", "Cliente borrado exitosamente");
        } else {
            redirectAttributes.addFlashAttribute("error", "Error al borrar el cliente");
        }
        return "redirect:/clientes"; // Adjust as needed
    }

    @PostMapping("/editar-mensaje")
    public String editarMensaje(@ModelAttribute Mensaje mensaje, RedirectAttributes redirectAttributes) {
        if (mensajeService.editarMensaje(mensaje)) {
            redirectAttributes.addFlashAttribute("success", "Mensaje editado exitosamente");
        } else {
            redirectAttributes.addFlashAttribute("error", "Error al editar el mensaje");
        }
        return "redirect:/mensajes"; // Adjust as needed
    }

    @PostMapping("/borrar-mensaje/{id}")
    public String borrarMensaje(@PathVariable int id, RedirectAttributes redirectAttributes) {
        if (mensajeService.borrarMensaje(id)) {
            redirectAttributes.addFlashAttribute("success", "Mensaje borrado exitosamente");
        } else {
            redirectAttributes.addFlashAttribute("error", "Error al borrar el mensaje");
        }
        return "redirect:/mensajes"; // Adjust as needed
    }
}

```

SPRINT III

REQ005

Prueba caja blanca de describa el requisito funcional

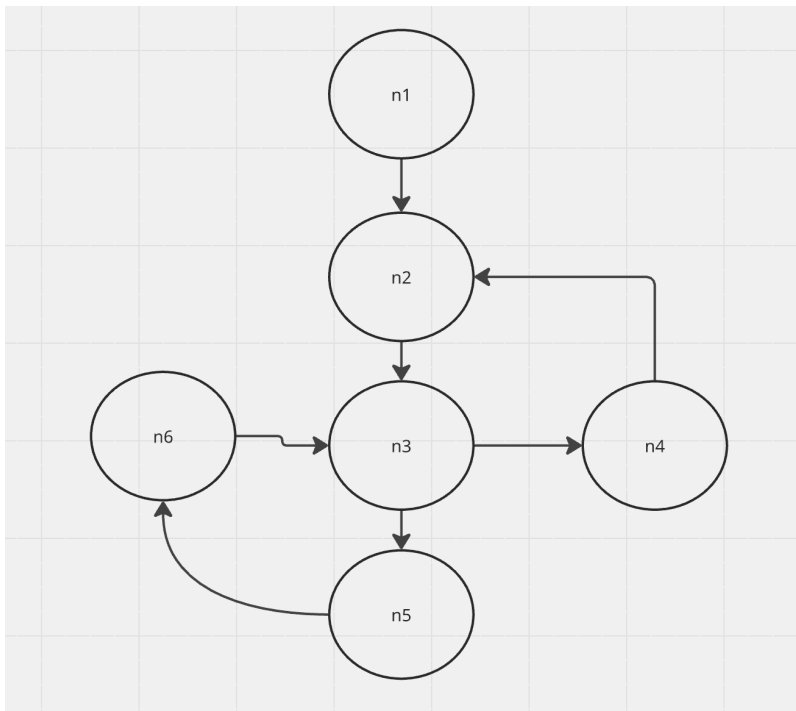
1. CÓDIGO FUENTE

2. DIAGRAMA DE FLUJO (DF)



5. GRAFO DE FLUJO (GF)

Realizar un GF en base al DF del numeral 2



6. IDENTIFICACIÓN DE LAS RUTAS (Camino básico)

Determinar en base al GF del numeral 4

RUTAS

R1: n1-n2-n3-n4

R2: n1-n2-n3-n4-n3-n5

R3: n1-n2-n3-n3-n5-n6-n3-n5

7. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = (3)+1$
 $V(G)=4$
- $V(G) = A - N + 2$

$$V(G) = 7 - 6 + 2 = 3$$

DONDE:

P: Número de nodos predicado

A: Número de aristas

N: Número de nodos

SPRINT III

REQ006

Prueba caja blanca de describa el requisito funcional

Realizar mensajes predefinidos para los clientes, basados en fechas conmemorativas.

1. CÓDIGO FUENTE

```
# Java
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.time.LocalDate;
import java.time.MonthDay;
import java.util.*;

@Service
public class FechasConmemorativasService {

    @Autowired
    private MensajeService mensajeService;

    private final Map<MonthDay, String[]> fechasConmemorativas = new HashMap<>();

    public FechasConmemorativasService() {
        inicializarFechasConmemorativas();
    }

    private void inicializarFechasConmemorativas() {
        // Ejemplos de fechas y mensajes
        fechasConmemorativas.put(MonthDay.of(1, 1), new String[]{"Año Nuevo", "¡Feliz Año Nuevo! Que este año venga cargado de bendiciones."});
        fechasConmemorativas.put(MonthDay.of(2, 14), new String[]{"Día de San Valentín", "¡Feliz Día de San Valentín! Celebra el amor y la amistad."});
        fechasConmemorativas.put(MonthDay.of(3, 8), new String[]{"Día de la Mujer", "¡Feliz Día de la Mujer! A todas las mujeres del mundo."});
        fechasConmemorativas.put(MonthDay.of(5, 10), new String[]{"Día de la Madre", "¡Feliz Día de la Madre! Gracias por tu amor incondicional."});
        fechasConmemorativas.put(MonthDay.of(6, 21), new String[]{"Día del Padre", "¡Feliz Día del Padre! Honramos a los padres ejemplares."});
        fechasConmemorativas.put(MonthDay.of(12, 25), new String[]{"Navidad", "¡Feliz Navidad! Que esta fecha especial traiga felicidad a tu familia."});
        // Agrega más fechas conmemorativas según necesites
    }

    public List<String[]> obtenerFechasProximas() {
        LocalDate hoy = LocalDate.now();
        List<String[]> fechasProximas = new ArrayList<>();

        for (int i = 0; i < 30; i++) { // Próximos 30 días
            LocalDate fecha = hoy.plusDays(i);
            MonthDay monthDay = MonthDay.from(fecha);

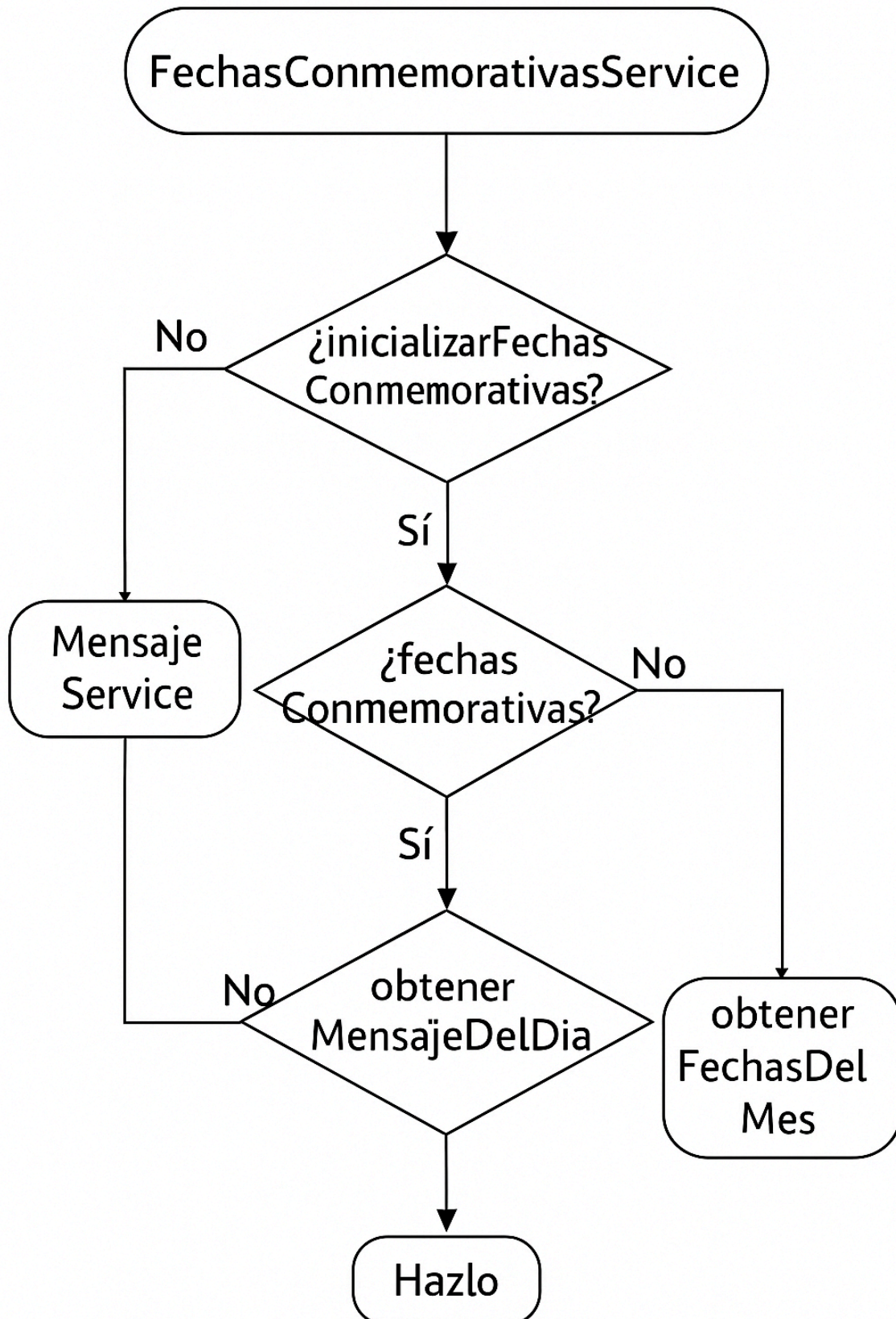
            if (fechasConmemorativas.containsKey(monthDay)) {
                String[] info = fechasConmemorativas.get(monthDay);
                fechasProximas.add(new String[]{
                    fecha.toString(),
                    info[0],
                    info[1]
                });
            }
        }

        return fechasProximas;
    }

    public String[] obtenerMensajeDelDia() {
        MonthDay hoy = MonthDay.from(LocalDate.now());
        return fechasConmemorativas.get(hoy);
    }

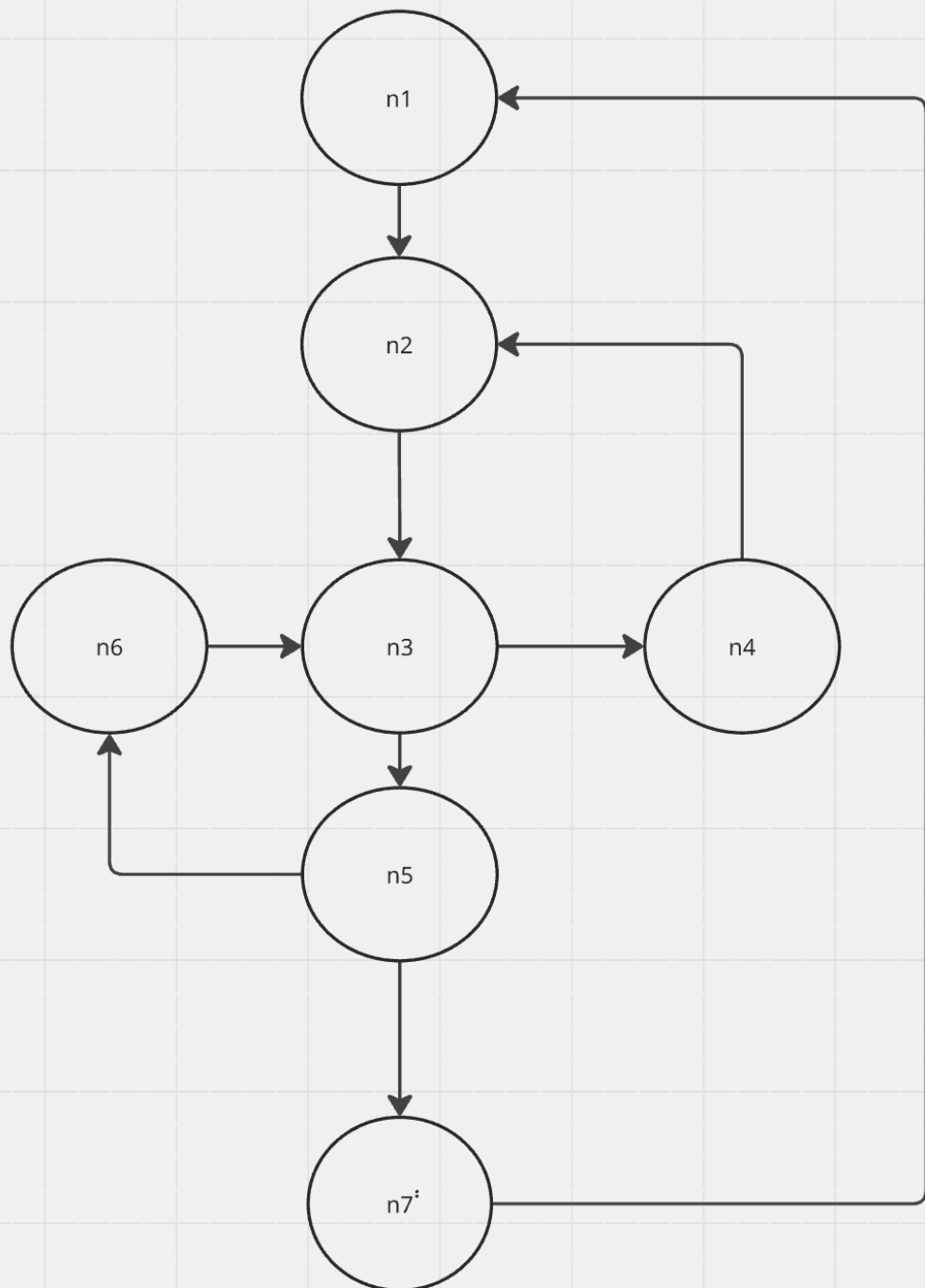
    public Map<String, Object> obtenerFechasDelMes() {
        Map<String, Object> resultado = new HashMap<>();
        resultado.put("fechasProximas", obtenerFechasProximas());
        resultado.put("mensajeDelDia", obtenerMensajeDelDia());
        return resultado;
    }
}
```


2. DIAGRAMA DE FLUJO (DF)



5. GRAFO DE FLUJO (GF)

Realizar un GF en base al DF del numeral 2



6. IDENTIFICACIÓN DE LAS RUTAS (Camino básico)

Determinar en base al GF del numeral 4

RUTAS

R1: n1-n2-n3-n4

R2: n1-n2-n3-n4-n3-n5

R3: n1-n2-n3-n3-n5-n6-n3-n5

R4: n1-n2-n3-n5-n7-n1-n2-n3-n5-n7

7. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = (3)+1$
 $V(G)=4$
- $V(G) = A - N + 2$
 $V(G) = 8 - 6 = 2$

DONDE:

P: Número de nodos prediado

A: Número de aristas

N: Número de nodos

SPRINT III

REQ007

Prueba caja blanca de describa el requisito funcional

Programar envíos automáticos en base a fechas registradas

1. CÓDIGO FUENTE

```
Java Copy

import org.springframework.scheduling.annotation.Scheduled;

// ...

public class FechasConmemorativasService {

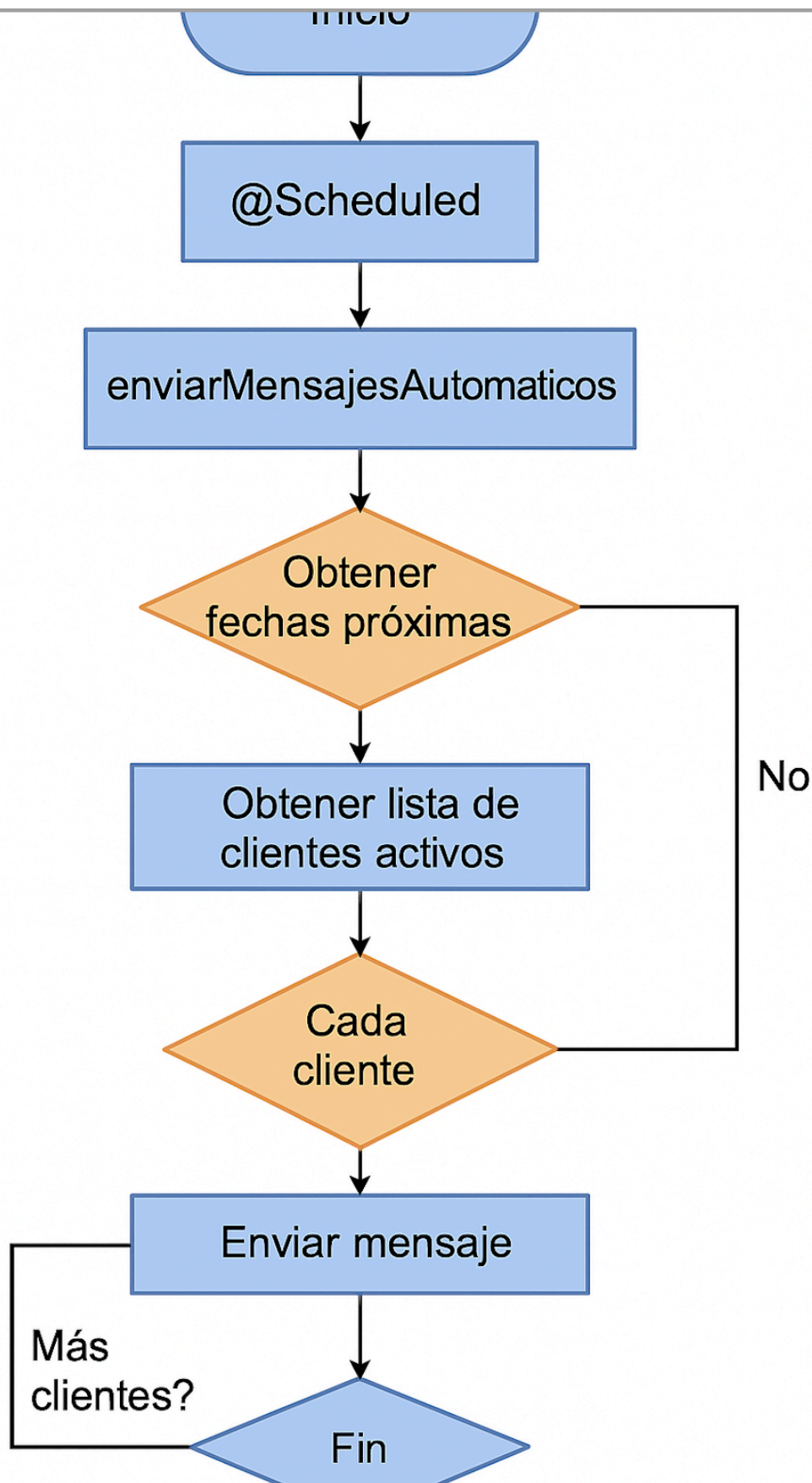
    @Autowired
    private MensajeService mensajeService;

    // Otros métodos y variables

    // Programación de envíos automáticos
    @Scheduled(cron = "0 0 9 * * ?") // Ejecuta todos los días a las 9 AM
    public void enviarMensajesAutomaticos() {
        List<String[]> fechasProximas = obtenerFechasProximas();
        for (String[] fechaInfo : fechasProximas) {
            String mensaje = fechaInfo[1]; // Mensaje del día
            // Lógica para enviar el mensaje a todos los clientes
            // Puedes hacer uso del MensajeService para manejar el envío
            for (int clienteId : obtenerListaClientesActivos()) { // Método que debes implementar para obtener los IDs de los
clientes activos
                mensajeService.enviarMensaje(clienteId, mensaje);
            }
        }
    }

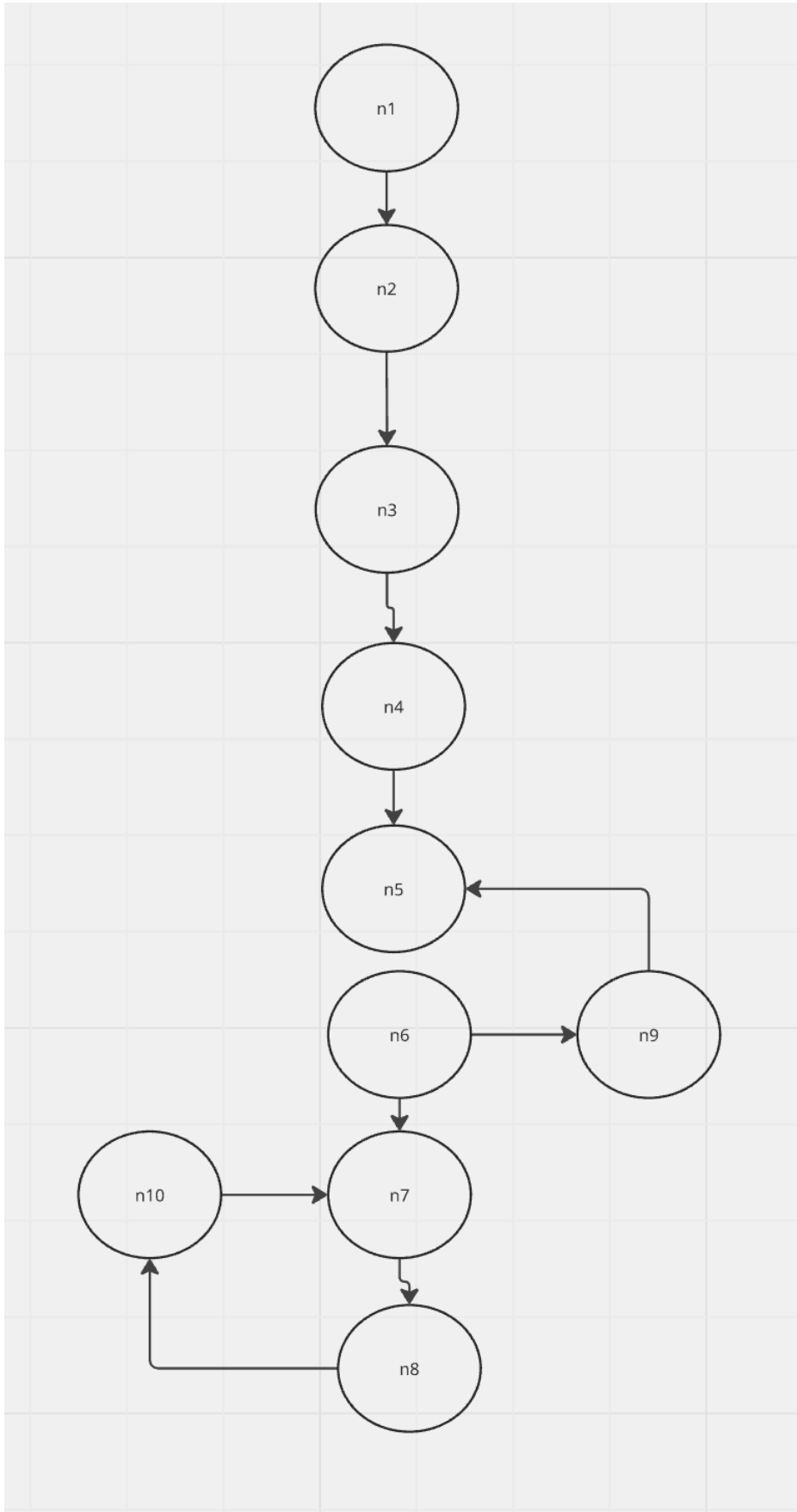
    private List<Integer> obtenerListaClientesActivos() {
        // Aquí debes implementar la lógica para obtener los IDs de los clientes activos
        // Esto puede ser una consulta a la base de datos o cualquier otra lógica que uses
        return new ArrayList<>(); // Completa con tu lógica específica
    }
}
```

2. DIAGRAMA DE FLUJO (DF)



5. GRAFO DE FLUJO (GF)

Realizar un GF en base al DF del numeral 2



6. IDENTIFICACIÓN DE LAS RUTAS (Camino basico)

Determinar en base al GF del numeral 4

RUTAS

R1: n1-n2-n3-n4-n5-n6-n7-n8

R2: n1-n2-n3-n4-n5-n9-n5-n6-n7-n8

R3: n1-n2-n3-n4-n5-n6-n7-n8-n10-n7-n8

7. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = (3)+1$
 $V(G)=4$
- $V(G) = A - N + 2$
 $V(G)= 9-10+2=1$

DONDE:

P: Número de nodos prediado

A: Número de aristas

N: Número de nodos